

SEMANTIC SEARCH

Comparing Word2Vec, SBERT & Hybrid Approaches

CS328 — Introduction to Data Science

MS MARCO v2.1 Passage Retrieval · 200K Queries · ~2M Passages

200K
queries

~2M
passages

3
designs

Why Semantic Search?

Keyword matching fails to capture meaning. We need models that understand intent.

01

Keyword Mismatch

Exact-match retrieval misses paraphrased queries — same meaning, different words.

02

Scale Problem

~2M passages in MS MARCO.
Brute-force comparison is computationally infeasible.

03

Contextual Queries

Natural language needs context:
'bank' means different things in different sentences.

04

Speed vs Accuracy

Production systems must balance retrieval quality with low query latency.

OUR GOAL

Design, implement, and evaluate three semantic retrieval systems on MS MARCO v2.1 — and identify the best trade-off between retrieval quality and query latency.

Microsoft MS MARCO v2.1

200,000

Training Samples

1,995,277

Total Passages

414,512

Word2Vec Vocab

384

SBERT Dimensions

305

Queries Evaluated

SAMPLE QUERY

"what was the immediate impact of the success of the manhattan project?"

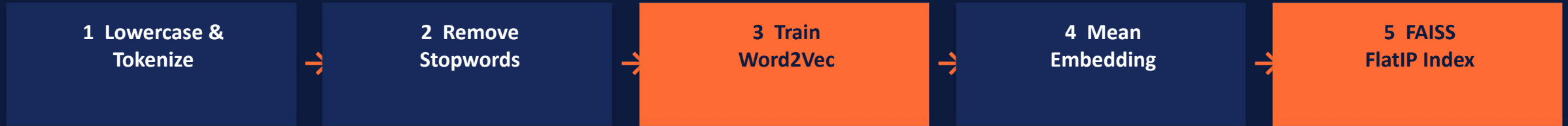
Data Loading

- › Hugging Face dataset: microsoft/ms_marco (v2.1 split)
- › 200,000 samples selected from the full training split
- › Each sample contains up to 10 passage candidates
- › is_selected flag marks relevant passages (ground truth)

Embedding Stats

- › 1,995,141 valid Word2Vec embeddings (300-dim, float32)
- › 1,995,277 SBERT embeddings (384-dim, normalized float32)
- › 195 of 500 evaluation queries skipped — no relevant passage
- › Fields: query, passages, query_type, answers, wellFormedAnswers

Word2Vec + Query Expansion



WORD2VEC MODEL

vector_size	300
window	5
min_count	2
epochs	5
vocabulary	414,512 tokens
storage	Memory-mapped .dat file

QUERY EXPANSION (QueryEncoder)

- › Preprocess query (same pipeline as corpus)
- › For each token: find top-N nearest Word2Vec neighbours
- › Filter by cosine similarity ≥ 0.5
- › Weight: original tokens $\times 1.0$, expanded $\times 0.7$
- › L2-normalize final averaged vector → FAISS search

global warming effects on sea level → greenhouse, ocean, seas, climatic, levels ...

SBERT — Sentence Transformer Bi-Encoder

MODEL: all-MiniLM-L6-v2

Architecture	6-layer MiniLM Transformer
Embedding Dim	384
Batch Size (encode)	512
Total Passages	1,995,277
Embedding File	sbert_embeddings.dat (float32)
Normalization	L2-normalized for cosine sim
FAISS Index	IndexFlatIP — exact search

SAMPLE RESULTS — "global warming effects on sea level"

- #1

score 0.808

There are two major effects of global warming: rise in temperature 3–5°C and sea levels by 25 m by 2100.
- #2

score 0.805

Melting glaciers will cause sea levels to rise 0.11–0.77 m by 2100 (IPCC 2001 report).
- #3

score 0.800

Effects of global warming: temperature increase and sea level rise of at least 25 m.
- #4

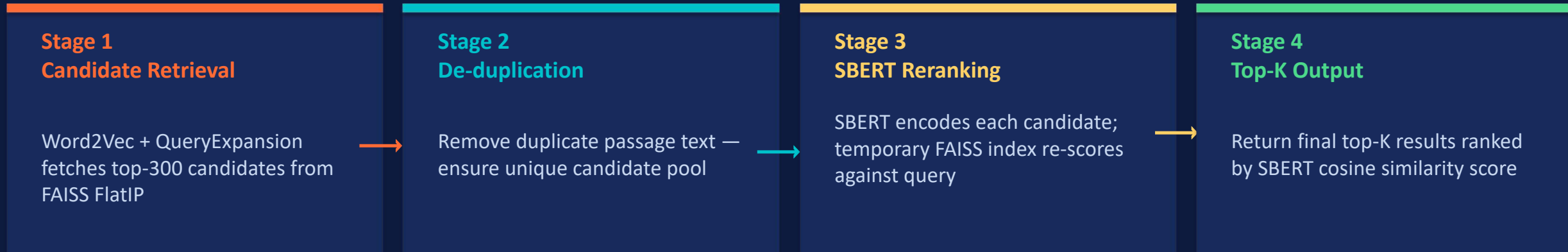
score 0.785

Antarctica ice melting; sea level may rise more than 1 metre by 2100.
- #5

score 0.779

Even limiting warming to 2°C — sea levels may still rise 6+ meters above current heights.

Hybrid: Word2Vec Retrieval + SBERT Reranking



KEY DESIGN DECISIONS

- › rerank_k=300 gives the reranker enough diversity without being too slow
- › Word2Vec expansion boosts coarse recall; SBERT then corrects ranking errors
- › Temporary per-query FAISS index is built only over the 300 candidates — fast and exact
- › Final scores come from SBERT — so ranking quality matches Design 2 for retrieved candidates

Why speed matters

Word2Vec is much faster than BERT.
Running BERT on 1M docs = very slow.
Running BERT on just 100 = totally fine!

Why accuracy improves

Word2Vec finds good candidates quickly.
BERT then re-ranks those 300 with better understanding — so final results are more relevant!

Metrics & Protocol

P@k

Precision at k

Out of the top k results, how many are relevant?
Higher value means fewer wrong results.

R@k

Recall at k

Out of all relevant passages, how many were found in top k results?
Higher value means fewer missed relevant results.

F1@k

F1 Score at k

It combines both precision and recall into one combine number

Higher is better

MRR@k

Mean Reciprocal Rank

Checks how early the first relevant result appears.
Higher value means correct result comes earlier.

NDCG@k

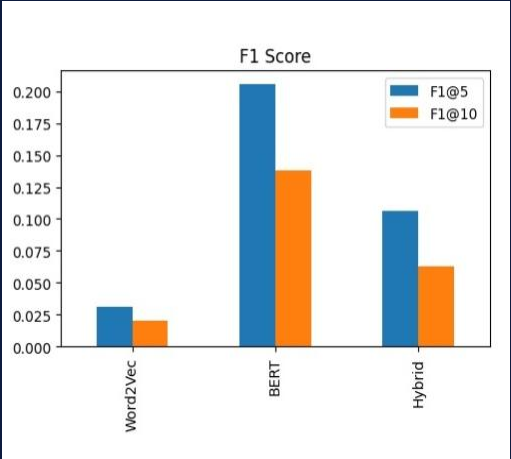
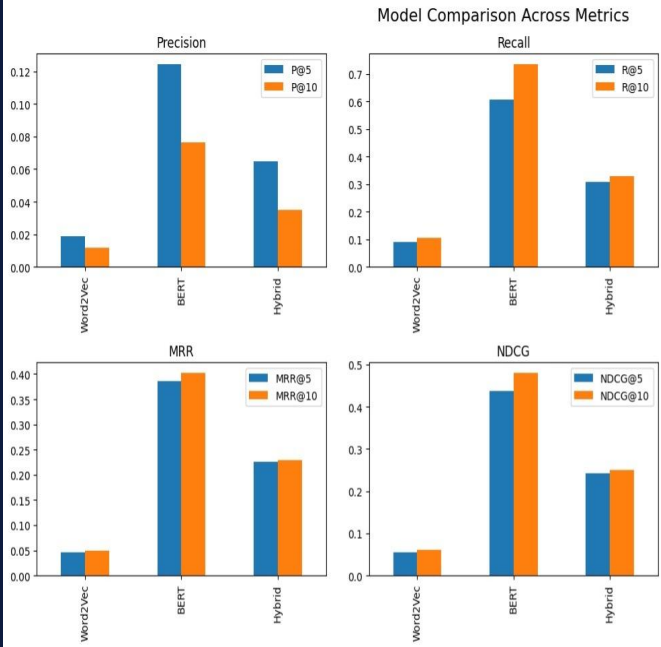
Normalized DCG

Measures ranking quality.
Gives higher score when relevant results appear at the top
Higher is better

Model Comparison — All Metrics

Method	P@5	R@5	F1@5	MRR@5	NDCG@5	P@10	R@10	MRR@10	NDCG@10
Word2Vec	0.019	0.090	0.031	0.047	0.056	0.012	0.106	0.049	0.061
SBERT	0.125	0.606	0.206	0.386	0.438	0.076	0.736	0.403	0.481
Hybrid	0.065	0.307	0.106	0.226	0.242	0.035	0.330	0.229	0.250

Low precision is normal in MS MARCO because only a few passages are relevant for each query. Recall is more important here. SBERT performs best: R@5 = 0.606 means it finds about 60% of relevant passages in the top 5 results, while Word2Vec finds only 9%. Hybrid improves retrieval quality compared to Word2Vec.



Query Expansion Size N

N	P@5	R@5	F1@5	MRR@5	NDCG@5	P@10	R@10
N = 3	0.022	0.106	0.037	0.063	0.072	0.015	0.142
N = 5	0.019	0.090	0.031	0.047	0.056	0.012	0.106
N = 10	0.013	0.060	0.021	0.036	0.041	0.010	0.093

N=3 is Optimal

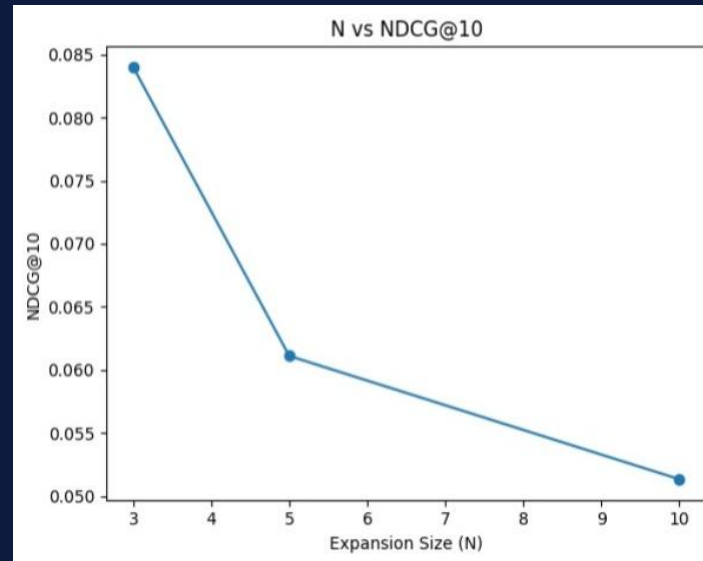
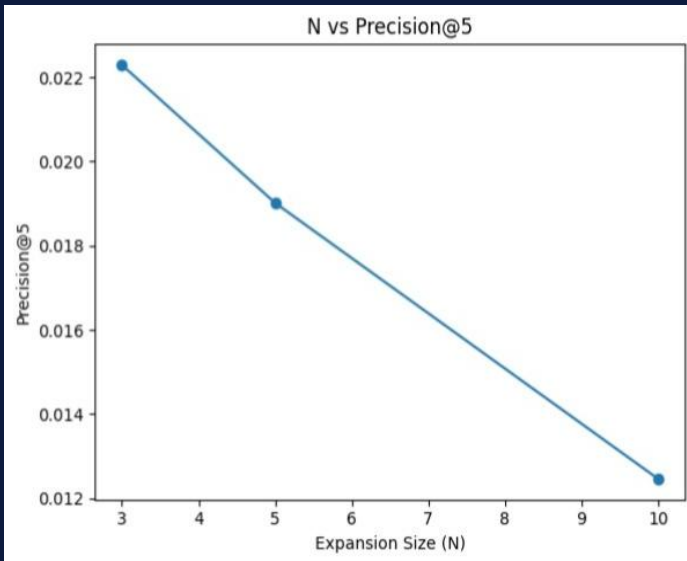
Smallest expansion still covers core synonyms. Score vectors stay focused on query meaning.

N=5 is Neutral

Default setting. Marginal improvement over N=10 but noticeable drop from N=3.

N=10 Hurts

Over-expanded query drifts toward loosely related terms. P@5 drops 41% vs N=3.



Takeaway

Expanding a query can help find more results, but it doesn't always improve quality in a steady way. Adding a small number of relevant terms (like N = 3) works better than adding too many (like N = 10), because it keeps the query closer to its original meaning instead of drifting away.

Query Latency Comparison

Test query: "what does deified mean" — k=10 retrieval

Word2Vec

380 ms

Fastest
Vector lookup + FAISS FlatIP.
No neural encoding at query time.

Hybrid

527 ms

Middle
Word2Vec retrieve + SBERT encode.
+147 ms overhead vs Word2Vec.

SBERT

541 ms

Slowest
Full transformer encode per query.
Pays accuracy tax in compute.

Speed: Word2Vec < Hybrid < SBERT | Accuracy: SBERT > Hybrid > Word2Vec

Limitations and Conclusions

Word2Vec

Ignores word order and sentence structure because it averages word vectors.

Cannot handle multiple meanings of a word (e.g., “bank”) since each word has only one vector.

Large query expansion may add irrelevant words and create noise.

SBERT

Slow for real-time use because each query needs neural encoding

Not fine-tuned on MS MARCO, so domain matching may be limited.

Requires large memory to store embeddings for millions of passages.

Exact search (FlatIP) scales poorly for very large datasets.

Hybrid Pipeline

Overall performance depends on Word2Vec first-stage retrieval.

If relevant passages are missed in stage 1, reranking cannot recover them.

Adds extra latency while still remaining below SBERT accuracy.

Two-stage pipeline is more complex to implement and maintain.

CONCLUSION

SBERT is most accurate but slow

Word2vec is fast but less accuracy

Small query expansion works best too many extra words reduce

Hybrid → fast + accuracy(balance)

FIASS makes it scalable → fast search even on millions of document

THANK YOU!

CS328 — Introduction to Data Science

Semantic Search on MS MARCO · Word2Vec · SBERT · Hybrid